



Phases of a Compiler and Their Role

Q1 — Understanding, Application, Analysis, Solution Design and Clarity of Compiler Phases

Presented by: ROHAN P T (192424112)

Compiler Design — Applied Concepts Assignment

OVERVIEW OF THE COMPILATION PROCESS

- A compiler translates high-level source code into equivalent target (machine or assembly) code.
- Compilation is organized into two broad parts: Analysis (front-end) and Synthesis (back-end).
- The process runs through six well-defined phases, each performing a specific transformation.
- A shared symbol table and error-handling routine support every phase throughout compilation.
- Understanding each phase clarifies exactly where and why a program can fail to compile.

6 Phases

Lexical to Code Generation

2 Stages

Analysis + Synthesis

Symbol Table

Shared Across All Phases

UNDERSTANDING OF CASE: THE SIX COMPILER PHASES

1. Lexical Analysis

Scans source code and groups characters into meaningful tokens (keywords, identifiers, operators, literals).

2. Syntax Analysis

Arranges tokens into a parse tree / AST by checking them against the language's grammar rules.

3. Semantic Analysis

Verifies meaning — type checking, scope resolution, and consistency of declarations and usage.

4. Intermediate Code Generation

Converts the validated AST into a machine-independent intermediate representation (e.g., three-address code).

5. Code Optimization

Improves the intermediate code for speed, memory, or power without changing program behaviour.

6. Code Generation

Maps optimized intermediate code to the target machine's instruction set, producing executable code.



ROLE OF EACH PHASE IN TRANSFORMING SOURCE TO MACHINE CODE



Phase	Input	Output	Role
Lexical Analysis	Source code (chars)	Token stream	Removes whitespace/comments; forms tokens
Syntax Analysis	Token stream	Parse tree / AST	Checks grammatical structure of the program
Semantic Analysis	Parse tree / AST	Annotated AST	Validates types, scope, and declarations
Intermediate Code Gen.	Annotated AST	Intermediate code (IR)	Produces machine-independent representation
Code Optimization	Intermediate code	Optimized IR	Reduces time, memory, and resource usage
Code Generation	Optimized IR	Target machine code	Generates final executable instructions

APPLICATION: DETECTING ERRORS PHASE BY PHASE

1

Lexical Errors

Invalid tokens, illegal characters, malformed numbers or unterminated strings (e.g. 5x, #abc, 'unterminated').

2

Syntax Errors

Missing semicolons, unmatched braces/parentheses, incorrect statement structure (e.g. missing ';' or ')').

3

Semantic Errors

Type mismatches, undeclared variables/functions, incompatible operand types, wrong argument counts.

Detecting errors at the earliest possible phase keeps diagnostics precise and avoids wasted downstream processing.



APPLICATION: PHASE-WISE ERROR IDENTIFICATION IN A C PROGRAM



```
int main() {  
    int x = 10  
    y = x + ;  
    if (x > 5) {  
        printf("%d", z);  
    }  
}
```

Line / Issue	Phase that Detects It	Reason
int x = 10 (no ';')	Syntax Analysis	Statement terminator missing — grammar violation
x + ; (empty operand)	Syntax Analysis	Incomplete expression after '+' operator
y (undeclared)	Semantic Analysis	Identifier used without prior declaration
z (undeclared)	Semantic Analysis	Identifier used without prior declaration

Key Insight

- Lexical, syntax, and semantic checks act as sequential filters — this snippet never reaches optimization or code generation until every earlier error is fixed.



ANALYSIS: PHASE DEPENDENCY AND EARLY-STAGE ERRORS

Phase-to-Phase Dependency

- Each phase consumes the previous phase's output — a syntax tree cannot form without a valid token stream.
- The symbol table is built and updated incrementally, so later phases (semantic analysis onward) rely on entries created earlier.
- An incorrect or incomplete earlier phase produces malformed input for the next, cascading errors downstream.

Why Early-Stage Errors Halt Compilation

- Lexical or syntax errors mean no valid parse tree exists, so semantic checks have nothing reliable to analyze.
- Compilers typically stop (or flag and skip) further phases once structural errors are found, avoiding misleading downstream diagnostics.
- This fail-fast behaviour saves computation and keeps error messages traceable to their true root cause.

Takeaway

Compilation is a strictly ordered pipeline: correctness of each phase is a precondition for the next. Front-loading validation in lexical and syntax analysis prevents cascading, harder-to-diagnose semantic or runtime failures.

SOLUTION DESIGN: ROBUSTNESS THROUGH RECOVERY AND MODULARITY

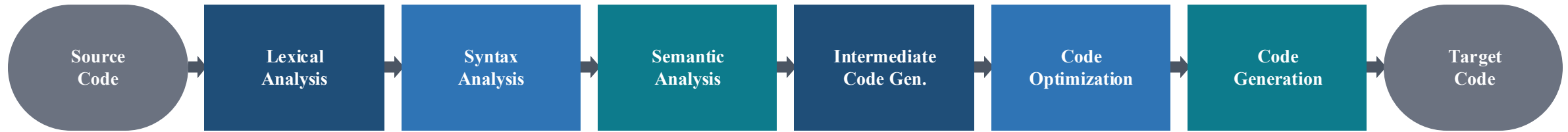
Error Recovery Techniques

- Panic mode: discard input tokens until a clear synchronizing symbol (e.g. ';') is found.
- Phrase-level recovery: perform local correction, such as inserting a missing token.
- Error productions: extend the grammar to anticipate and recognise common mistakes.
- Global correction: compute the minimal edit distance to a valid program (mainly theoretical).

Modular Compiler Design

- Separate front-end (analysis) from back-end (synthesis) so each can evolve independently.
- Well-defined interfaces between phases make individual modules easier to test and debug in isolation.
- A common intermediate representation lets the same front-end retarget to multiple back-ends (portability).
- Modules can be swapped or upgraded (e.g. a new optimizer) without rewriting the rest of the compiler.

CLARITY: FLOWCHART OF THE COMPILER PHASE SEQUENCE



Symbol Table & Error Handler — accessed and updated by every phase throughout compilation

- Analysis phases (lexical → semantic) break the program down and validate it; synthesis phases (intermediate code → code generation) build the target program back up.
- The pipeline is strictly sequential: output of one phase is the sole input to the next.



CLARITY: STEP-BY-STEP TRACE OF A SINGLE STATEMENT

Source statement: `a = b + c * 2;`

1. Lexical Analysis

Tokens: `id(a) = id(b) + id(c) * num(2) ;`

2. Syntax Analysis

Parse tree built from grammar:
assignment $\rightarrow$ id '=' expr ';' with '*'
binding tighter than '+'.

3. Semantic Analysis

Confirms `a`, `b`, `c` are declared and type-compatible; annotates tree with types.

4. Intermediate Code Generation

```
t1 = c * 2
t2 = b + t1
a = t2
```

5. Code Optimization

Constant/strength adjustments applied where possible; redundant temporaries removed.

6. Code Generation

```
MOV R1, c    MUL R1, #2    ADD R1, b
MOV a, R1
```

CONCLUSION & KEY TAKEAWAYS

- Compilation proceeds through six ordered phases, each transforming the program closer to executable machine code.
- Every phase depends on the correctness of the one before it, making early error detection critical.
- Lexical and syntax errors halt compilation early; semantic checks catch deeper meaning-level mistakes.
- Error-recovery strategies and a modular front-end/back-end design make compilers more robust and maintainable.
- Flowcharts and worked traces make the otherwise abstract phase sequence concrete and easy to reason about.

6
Sequential Phases

3
Error-Detecting Stages (Lexical, Syntax, Semantic)

Modular
Design Enables Easier Debugging



THANK YOU!

Phases of a Compiler and Their Role